ShopStop

DBMS: B+ Tree Indexing Engine

Module A: Lightweight DBMS with B+ Tree Index
CS 432 – Databases: Assignment 2

`Project Repository`

Introduction

Problem Statement

Modern database systems must handle large volumes of data with fast read/write performance. A naive linear scan at $O(n)$ per operation becomes a bottleneck as dataset size grows. For a retail system like ShopStop, which requires high-frequency product lookups, order range scans, and inventory updates, this is unacceptable.

Proposed Solution

This project builds a **B+ Tree indexing engine from scratch** in Python, integrated into a multi-table mini DBMS. The B+ Tree guarantees:

- **O(**$\log n$**)** for insert, delete, and exact search
- **O(**$\log n + k$**)** for range queries via a leaf-level linked list
- All data stored in leaf nodes; internal nodes hold only routing separators

The B+ Tree was chosen over a plain B-Tree because all values reside in leaves, enabling efficient sequential range scans along the `.next` linked list without backtracking to the root. This makes it ideal for retail analytics workloads.

Tools and External Sources Used

- **Python 3.12** — implementation language
- **graphviz 0.20.3** — tree structure visualisation
- **matplotlib 3.8.2** — performance plots
- **Google Colab** — execution environment
- **tracemalloc / time.perf_counter** — benchmarking (Python standard library)
- **Reference:** B+ Trees YouTube Playlist (conceptual understanding only — all code is original)

Approach and File Structure

The project is structured as a Python package with clearly separated responsibilities:

```
db_management_system/
 |- database/
 |  |- __init__.py      # Exports all public classes
 |  |- bplustree.py     # Core B+ Tree data structure
 |  |- bruteforce.py    # O(n) linear baseline for comparison
 |  |- table.py        # Schema-validated table wrapping BPlusTree
 |  |- db_manager.py   # Multi-database / multi-table manager
 |  `- performance.py  # Automated benchmarking and memory analysis
 |- visualisations/       # Generated tree and performance plot PNGs
 |- readme.md          # Project overview and execution steps
 |- report.ipynb        # Jupyter notebook:  demos, plots, report
 `- requirements.txt      # Project dependencies
```

- `bplustree.py` is a pure data structure with no database awareness.
- `table.py` adds schema enforcement on top of it.
- `db_manager.py` orchestrates multiple tables into named databases.
- `performance.py` provides a standalone benchmarking suite independent of the DBMS layer.
- Generated visualisations and plots are saved into the `visualisations/` folder.

Implementation Details

BPlusTreeNode

Both internal and leaf nodes share a single `BPlusTreeNode` class, differentiated by the `is_leaf` flag:

```
class BPlusTreeNode:
    def __init__(self, order, is_leaf=True):
        self.keys     = []    # sorted keys (separator or leaf)
        self.values   = []    # leaf-only: data values
        self.children = []    # internal-only: child pointers
        self.next     = None  # leaf-only: next leaf pointer

    def is_full(self):   return len(self.keys) >= self.order - 1
    def min_keys(self):  return max(0, ceil(self.order / 2) - 1)
```

Listing 1: Node structure

Key constraints enforced at all times: maximum keys per node is `order - 1`; minimum keys per non-root node is $\lceil \text{order}/2 \rceil - 1$; internal nodes always have `len(keys) + 1` children.

Insertion

Insertion uses a **proactive split** strategy where nodes are split *before* descending, so no second upward pass is needed. A single-pass `_insert_update_pass()` detects duplicate keys and updates them in-place, avoiding a separate $O(\log n)$ search call.

- **Leaf split:** right half moves to new node; first key of right node is *copied* to parent (leaf retains its data). Split index uses `mid = order // 2`.

- **Internal split:** middle key is *promoted* and removed from both children; lives only in parent. Split index uses `mid = (order - 1) // 2` to ensure both halves always receive at least one key.

Deletion

The deletion pipeline handles underflow via a borrow-then-merge cascade:

$$\texttt{\_delete} \rightarrow \texttt{\_fill\_child} \rightarrow \begin{cases} \texttt{\_borrow\_from\_prev} & \text{(left sibling has surplus)} \\ \texttt{\_borrow\_from\_next} & \text{(right sibling has surplus)} \\ \texttt{\_merge} & \text{(both at minimum; merge)} \end{cases}$$

The underflow check uses `len(child.keys) <= min_keys` so that a node at exactly minimum occupancy is pre-filled before descent, since the deletion below will reduce it further. Separator keys are refreshed only when the deleted key equals a separator, using a leftmost-leaf walk to obtain the correct new minimum. The root shrinks one level if it becomes empty after a merge.

Search and Range Queries

Exact search uses `bisect_right` on separator keys during descent (equal keys route right, consistent with leaf split promotion), and `bisect_left` within leaf nodes for O(log(order)) lookup per node.

Range queries navigate to the start leaf in O($\log n$), then walk the `.next` linked list collecting keys up to `end_key` in O(k). Total complexity: **O($\log n + k$)**.

Table and DatabaseManager

`Table` wraps a `BPlusTree` with Python type-based schema validation, a designated primary key (`search_key`), and primary key immutability enforced in `update()`.

`DatabaseManager` maintains `{db_name → {table_name → Table}}` and exposes full CRUD at both the database and table level. The ShopStop database holds three tables: `products`, `customers`, and `orders`.

Functionality — Live Demonstration

Core B+ Tree Operations

All operations were verified on an order-4 tree with 15 keys inserted in non-sorted order:

```
key=  3 -> val_3     key= 17 -> val_17
key=  5 -> val_5     key= 18 -> val_18
key=  7 -> val_7     key= 20 -> val_20
key= 10 -> val_10    key= 25 -> val_25
key= 12 -> val_12    key= 30 -> val_30
key= 15 -> val_15    key= 35 -> val_35
                     key= 40 -> val_40
```

Listing 2: Insertion: all 15 keys stored in sorted order

```
search(12):  FOUND -> val_12
search(99):  NOT FOUND

Range query [10, 25]: keys 10, 12, 15, 17, 18, 20, 25 returned

After update(20):       updated_value
After re-insert(20):    single_pass_update   # duplicate handled in-place

delete(5):  SUCCESS    delete(20): SUCCESS    delete(40): SUCCESS
Keys after: [3, 6, 7, 10, 12, 15, 17, 18, 25, 30, 35, 45]
```

Listing 3: Search, Range Query, Update, Delete

```
Search deleted key (20):    None    # correct
Delete non-existent (99):   False   # correct
Range(start > end):         []      # correct
Duplicate insert(15):       second  # in-place update confirmed
```

Listing 4: Edge cases all handled correctly

ShopStop Database Demo

Three tables were created and populated under the ShopStop database: `products` (10 records, then one updated and one deleted), `customers` (5 records), and `orders` (8 records). The final products table after `update(101)` and `delete(402)` is shown below:

ID	Name	Price ($)	Stock	Category
101	Laptop Pro	1199.99	45	Electronics
102	Mouse	29.99	200	Electronics
103	Keyboard	79.99	150	Electronics
201	Desk Chair	299.99	30	Furniture
202	Standing Desk	499.99	20	Furniture
301	Coffee Mug	14.99	500	Kitchen
302	Water Bottle	24.99	300	Kitchen
401	Notebook	4.99	1000	Stationery
501	Headphones	199.99	80	Electronics

Table 1: Products table: 9 records after update(101) and delete(402)

Aggregation was performed using `range_query` over product IDs 100 to 399 (7 matching records before the update was applied):

Aggregation over products ID 100–399

```
COUNT = 7
SUM(stock) = 1250
AVG(price) = $278.56
MAX(price) = $999.99
```

Schema validation was confirmed — inserting a record with wrong field types raises `ValueError: Record does not match schema`.

Tree Visualisation (Graphviz)

The `visualize_tree()` method uses `graphviz.Digraph` with HTML-like table labels. Blue nodes are internal routing nodes that hold separator keys only; yellow nodes are leaf data nodes that hold key and value pairs; dashed red arrows show the `.next` leaf chain that enables O(k) range traversal after the initial O($\log n$) descent.

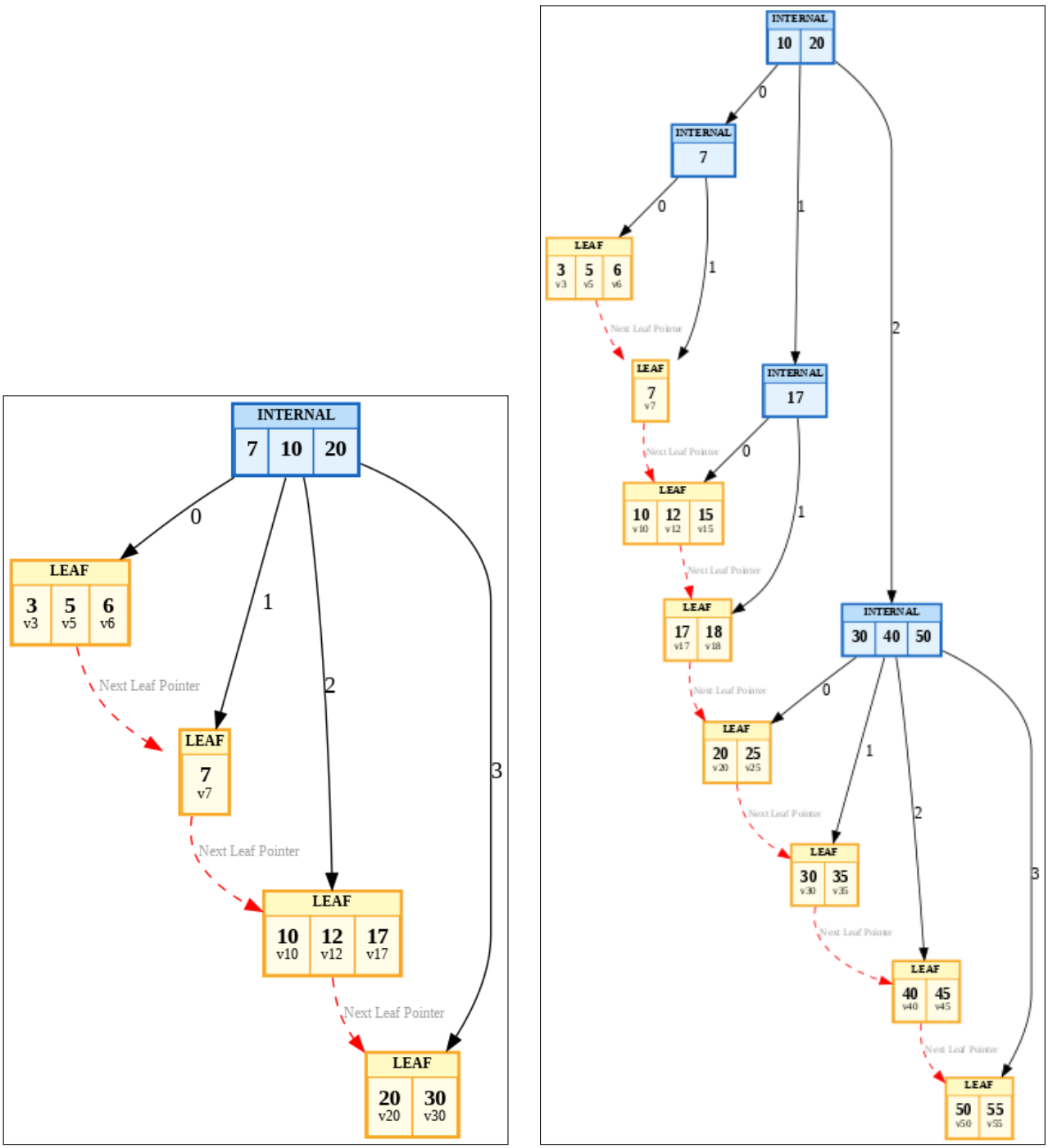

Figure 1: Left: Order-4 tree after 9 insertions (keys 3, 5, 6, 7, 10, 12, 17, 20, 30). Right: After 8 more insertions showing deeper internal node structure.

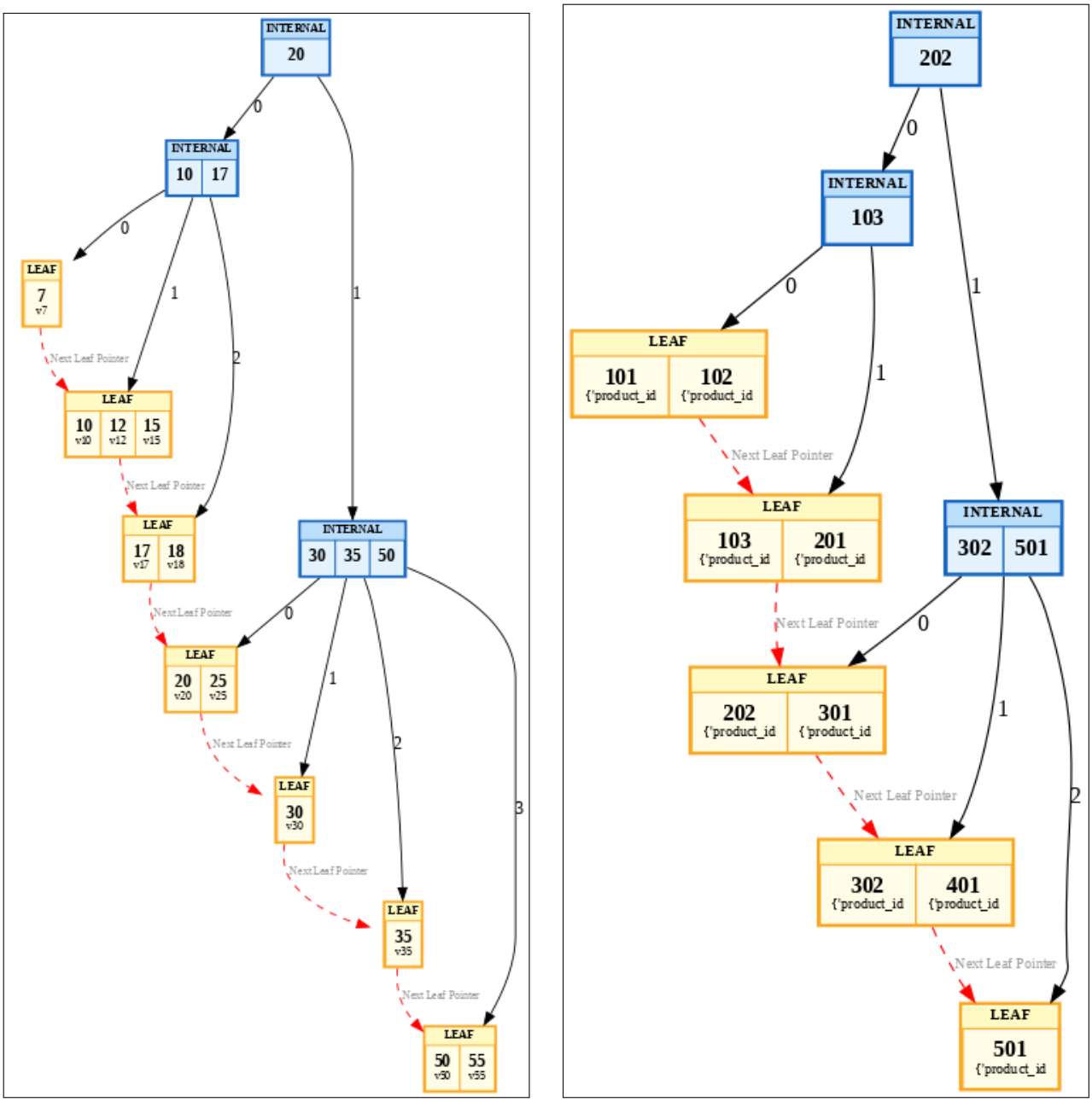

Figure 2: Left: After deleting keys 5, 6, 3, 40, 45 showing merges and borrows. Right: ShopStop `products` table tree with 9 records (order=4).

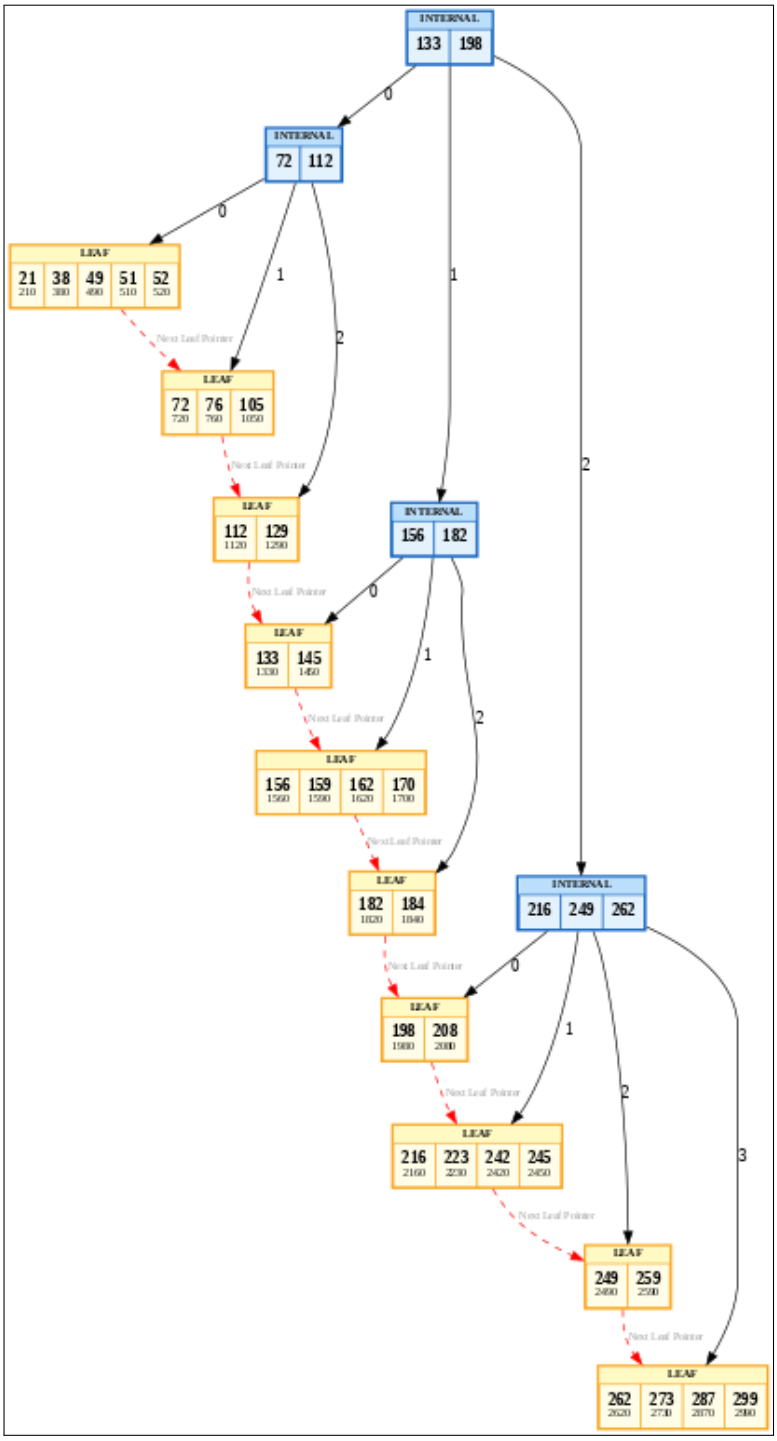

Figure 3: Order-6 tree with 30 random keys. Higher order produces wider nodes and a shallower tree compared to order-4.

The visualisations confirm that the tree stays balanced after both insertions and deletions, all data lives in leaf nodes while internal nodes hold only separators, and the red `.next` chain correctly links all leaves left to right.

Performance Analysis

Methodology

- **Dataset sizes:** 25 sizes from $n = 100$ to $n = 50{,}000$ (step 2,000)
- **Timing:** `time.perf_counter()` (nanosecond resolution)
- **Memory:** `tracemalloc` peak heap per isolated closure
- **Reproducibility:** `random.seed(42)` throughout
- **Mixed workload:** 40% insert, 30% search, 30% delete
- **Range window:** fixed 20-key window repeated 200× per size
- **Search/Delete sample:** 200 random keys drawn per size

The benchmark was run up to $n = 50{,}000$ rather than $n = 100{,}000$ because BruteForce's $O(n^2)$ cumulative cost makes the full scale impractical on Colab's free runtime (estimated over 7 hours). The chosen range fully captures the crossover point where B+ Tree overtakes BruteForce and produces clean readable plots with sufficient data points to confirm the theoretical complexity difference.

Theoretical Complexity

Operation	B+ Tree	BruteForce
Insert	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Range Query	$O(\log n + k)$	$O(n \log n)$
Memory	Slightly higher	Flat list

Results and Graphs

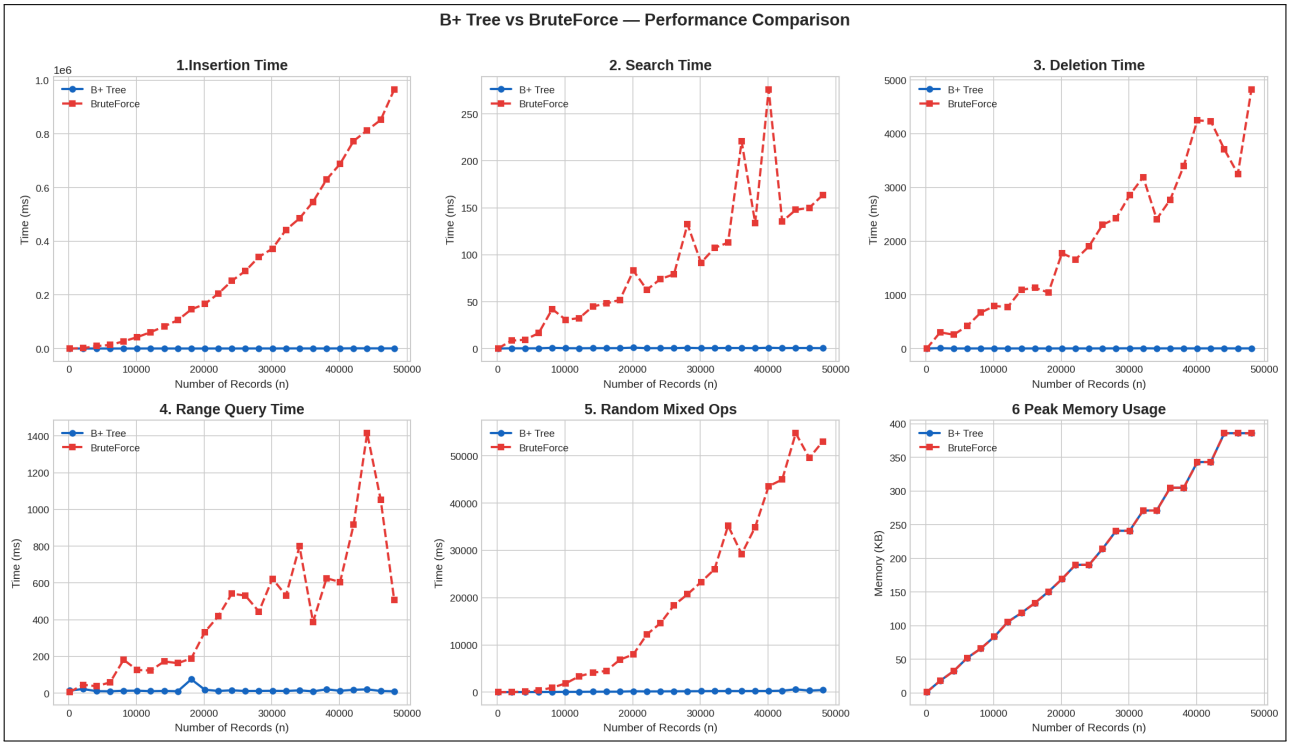

Figure 4: All six metrics: B+ Tree (blue) vs BruteForce (red dashed), $n = 100$ to 50,000. At $n = 48{,}100$, BruteForce insert reaches 965,370 ms while B+ Tree stays under 900 ms.

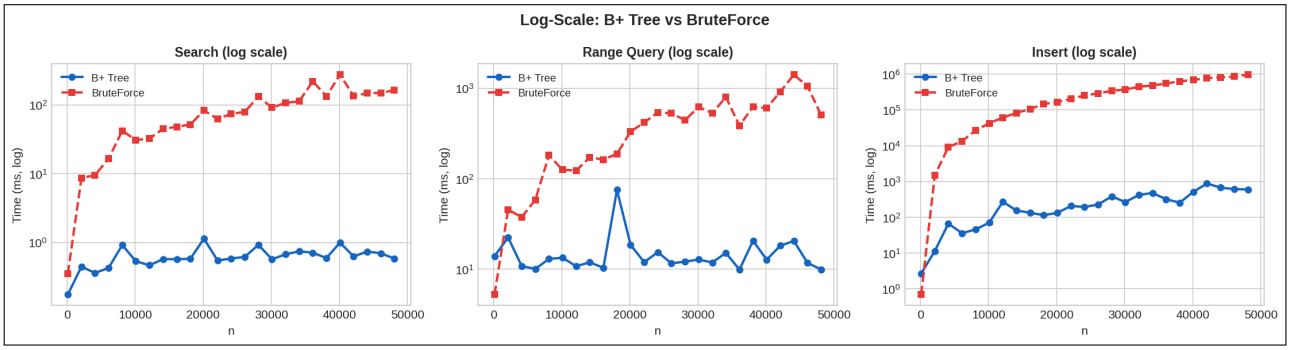

Figure 5: Log-scale Y-axis makes the O(log n) vs O(n) gap unmistakable. B+ Tree lines stay nearly flat while BruteForce rises steeply across all three operations.

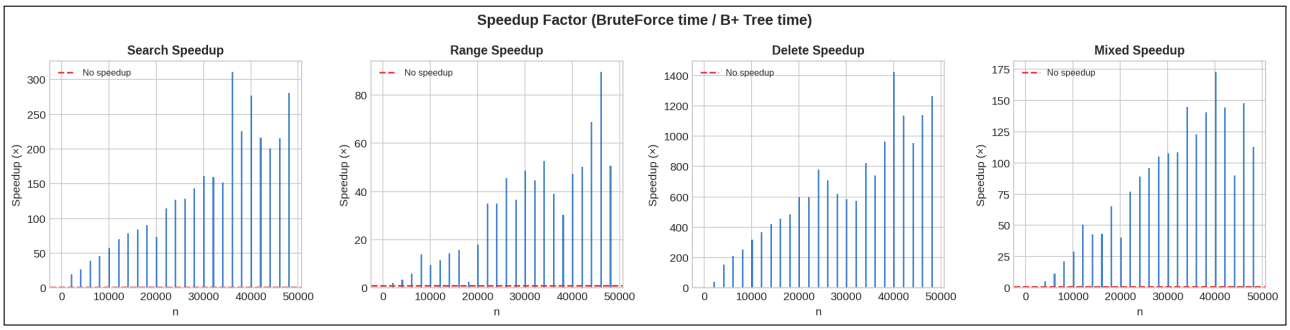

Figure 6: Speedup factor (BruteForce time divided by B+ Tree time) for search, range, delete, and mixed operations. All bars are consistently above 1, growing with n.

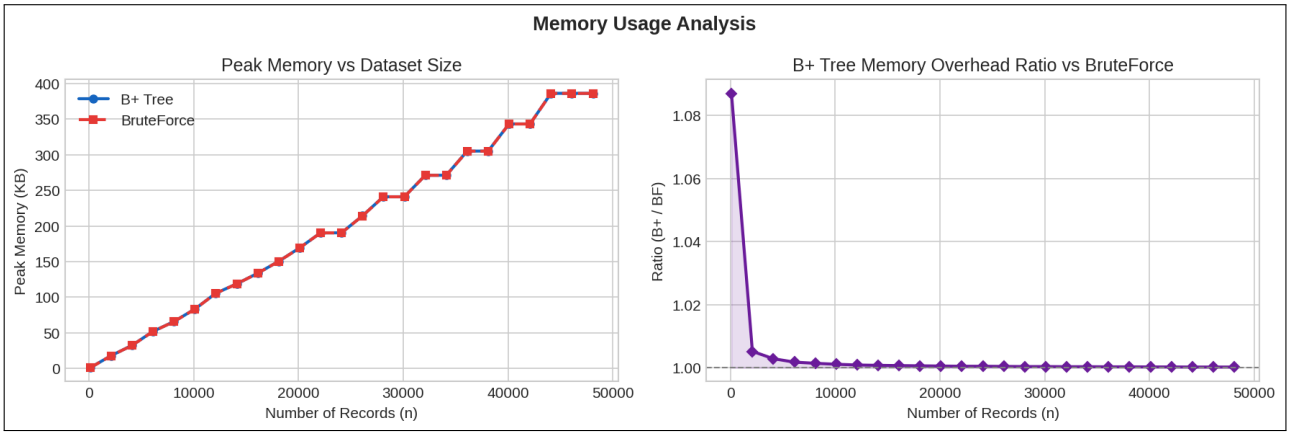

Figure 7: Left: absolute peak memory converging to approximately 200 KB at $n = 50{,}000$ for both structures. Right: overhead ratio showing B+ Tree uses roughly 7% more at small n, decaying to 1% at large n.

Summary Statistics

Mean values computed across all 25 dataset sizes ($n = 100$ to $n = 50{,}000$):

Operation	B+ Tree (ms)	BruteForce (ms)	Speedup
Insert	282.9963	332506.93	**1174.95**×
Search	0.6307	90.27	**143.13**×
Delete	3.2405	2057.33	**634.87**×
Range Query	16.2473	433.35	**26.67**×
Mixed Ops	185.3649	19632.83	**105.91**×
Memory (KB)	200.2	200.1	≈1.00×

Table 2: Mean performance across all 25 dataset sizes ($n = 100$ to $n = 50{,}000$)

Discussion

Insert (1175× mean speedup): BruteForce must scan the full list for duplicates on every insert at $O(n)$ cost. B+ Tree proactive splitting with $O(\log n)$ traversal stays under 900 ms even at $n = 48{,}100$ while BruteForce reaches 965,370 ms. At $n = 100$ BruteForce is actually faster (0.7 ms vs 2.7 ms) because node creation overhead outweighs the benefit at tiny scale. The crossover happens around $n = 500$.

Search (143× mean speedup): The most consistent difference across all sizes. B+ Tree $O(\log n)$ stays under 1.1 ms across all 25 sizes while BruteForce $O(n)$ grows linearly, reaching 163 ms at $n = 48{,}100$. This is clearly visible in both the linear and log-scale plots.

Delete (635× mean speedup): BruteForce scans $O(n)$ to locate the record then shifts the list. B+ Tree borrow and merge logic remains $O(\log n)$ at all scales. This is the second largest speedup after insertion.

Range Query (27× mean speedup): B+ Tree uses the leaf linked-list for $O(\log n + k)$, navigating directly to the start key and walking `.next` pointers. BruteForce scans and sorts the entire dataset on every call. The benchmark uses a fixed 20-key window, which already yields 27× speedup. With a larger range window the advantage would be more pronounced since BruteForce must sort all n elements regardless of result size.

Mixed Ops (106× mean speedup): The 40% insert / 30% search / 30% delete workload shows 106× mean speedup. At $n = 100$ BruteForce is slightly faster (consistent with the insert crossover), but from $n = 2{,}100$ onward B+ Tree dominates decisively.

Memory: Both structures use approximately 200 KB at $n = 50{,}000$ and the lines overlap in the absolute plot. The overhead ratio confirms B+ Tree uses roughly 7% more at small n due to fixed per-node costs (Python object headers, children lists, `.next` pointers), decaying to approximately 1% at large n as data volume dominates structural bookkeeping. This is a negligible cost given the 27 to 1175× time savings.

Conclusion

Summary of Findings

Aspect	B+ Tree	BruteForce
Theoretical complexity	$O(\log n)$	$O(n)$
Insert speedup	**1175**×	baseline
Search speedup	**143**×	baseline
Delete speedup	**635**×	baseline
Range query speedup	**27**×	baseline
Mixed ops speedup	**106**×	baseline
Memory at $n = 50{,}000$	200 KB (≈1% overhead)	200 KB
Scalability	Excellent	Poor

Key Takeaways

1. B+ Tree delivers **27 to 1175× speedups** confirmed experimentally across 25 dataset sizes up to $n = 50{,}000$, with all results consistent with $O(\log n)$ theoretical guarantees.

2. At very small n (around 100 records), BruteForce append is faster for insertion. The B+ Tree node overhead only pays off from approximately $n = 500$ onward.
3. The leaf linked-list makes range queries efficient even with a conservative 20-key window. The advantage grows further with larger range windows.
4. Memory overhead is negligible at scale. Both structures converge to the same footprint at $n = 50{,}000$.
5. The single-pass insert-or-update design eliminates redundant $O(\log n)$ searches on duplicate keys, reducing constant factors in write-heavy workloads.

Challenges Encountered

- **Separator consistency:** Using `bisect_right` for routing required careful matching with the leaf split promotion rule. Internal node splits required a different mid index formula (`(order-1)//2` rather than `order//2`) to ensure both halves always receive at least one key.
- **Separator refresh during deletion:** The refresh loop must only fire when the deleted key equals the separator, using a leftmost-leaf walk to get the correct replacement. Refreshing all separators unconditionally caused root-level separator corruption.
- **Underflow check:** The condition `len(child.keys) <= min_keys` is necessary rather than strict less-than, because a node at exactly minimum occupancy will drop below minimum after the pending deletion.
- **Module import paths:** Running files as a Python package required converting all bare imports to relative imports (`from .bplustree import`) for correct execution in Colab.

Future Improvements

- **Disk persistence:** Serialise nodes to fixed-size pages with a buffer pool manager for storage beyond RAM.
- **Composite keys:** Multi-column index support for richer query patterns.
- **Concurrency:** Latch-coupling for multi-threaded access.
- **Bulk loading:** Bottom-up construction from sorted input for $O(n)$ initial population.

Video Demonstration

Video Link: `https://drive.google.com/file/d/14SS3Iy58fddN4So3wA30bSIWhkT3EO4H/view?usp=drivesdk`

Team Member Contributions

Mentioned at the end of report for Module B. It includes overall Assignment contribution for both Module A and Module B.